

Практическая работа №1. Знакомство с проектом: C#, .NET и Git

Дисциплина: Алгоритмизация и программирование

Инструменты: .NET SDK 10.0, VS Code + C# Dev Kit, Git Bash, аккаунт на GitHub

Цели работы

- Установить и проверить окружение для разработки на C# (.NET SDK, VS Code, Git).
- Понять, что такое компилятор и зачем нужен dotnet.
- Создать репозиторий на GitHub и связать с ним локальную папку на компьютере.
- Написать несколько небольших программ на C#: вывод текста, переменные, арифметика, строки, константы.
- Освоить базовый рабочий цикл Git: add → commit → push.
- Разобраться, из каких файлов состоит консольный проект .NET и что можно (а что нельзя) заливать в репозиторий.

Важно. В этом семестре мы делаем упор на **синтаксис языка C#** — переменные, условия, циклы, методы, коллекции, строки, файлы. Алгоритмы (поиск, сортировки, рекурсия) разберём подробно в конце курса, когда вы уже будете свободно писать код. Параллельно у вас идёт курс «Архитектура аппаратных средств» — там мы отдельно разбираем, как код превращается в машинные инструкции и что происходит в памяти; здесь мы такие темы трогать не будем, чтобы не дублироваться.

Теоретическая справка

Что такое компилятор и зачем он нужен

Вы уже писали код на Python — там программа выполняется **построчно** специальной программой-интерпретатором: она читает строку, тут же её выполняет, читает следующую.

C# устроен иначе. Прежде чем что-то запустить, специальная программа — **компилятор** — читает **весь** исходный код (.cs файлы) и переводит его в промежуточный машинно-независимый код, который затем выполняет среда .NET. Если в коде есть синтаксическая ошибка, компилятор откажется собирать программу **вообще** — она не запустится, даже если ошибка в строке, которая никогда не выполнится. В Python вы бы узнали об ошибке, только дойдя до неё во время выполнения.

Команда `dotnet run`, которую вы будете использовать почти на каждом занятии, на самом деле делает две вещи по очереди:

1. **Собирает** проект (компилирует `.cs` файлы) — эта стадия называется **build**.
2. **Запускает** получившуюся программу.

Более глубоко про то, во что именно превращается ваш код, говорим на курсе «Архитектура аппаратных средств» — там это в фокусе. Здесь для нас **dotnet run** пока просто «кнопка запустить».

Структура консольного проекта .NET

Когда вы создаёте проект командой `dotnet new console`, появляется несколько файлов и папок:

Файл / папка	Что это
Program.cs	Исходный код вашей программы — здесь вы пишете C#.
*.csproj	XML-файл настроек проекта: какая версия .NET, какие пакеты подключены.
bin/	Сюда компилятор кладёт скомпилированную программу (результат сборки).
obj/	Служебные промежуточные файлы сборки.

Важно: папки `bin/` и `obj/` — это **не исходный код**, а результат работы компилятора на конкретном компьютере. Их не нужно хранить в Git — они пересоздаются заново командой `dotnet run`. Для этого существует файл `.gitignore`.

Что такое Git и зачем он нужен

Git — система контроля версий: она запоминает историю изменений вашего кода в виде «снимков» (**коммитов**), к которым всегда можно вернуться. Это решает несколько проблем:

- Не нужно хранить папки `project_final`, `project_final2`, `project_final_ИСПРАВЛЕННЫЙ`.
- Можно посмотреть, что именно изменилось между версиями.
- Преподаватель видит **процесс** работы, а не только финальный результат.
- Это стандарт индустрии: без Git не обходится ни одна реальная команда разработчиков.

GitHub — облачный сервис, на котором хранится «удалённая» копия вашего репозитория.

Рабочий цикл Git

Весь семестр вы будете повторять один и тот же цикл, работа за работой:

- git add .** # 1. Пометить изменённые файлы к сохранению
- git commit -m "Описание"** # 2. Сохранить снимок (коммит) с сообщением
- git push** # 3. Отправить снимок на GitHub

Правило хорошего сообщения коммита: кратко и по делу, что именно сделано. "Lab01: first console app" — хорошо. "фывфыв" или "готово" — плохо, преподаватель не поймёт, что произошло.

Часть 1. Установка и проверка окружения

Шаг 1. Проверка .NET SDK

Откройте **Git Bash** и выполните:

dotnet --version

Ожидаемый результат — версия вида 10.0.xxx. Если команда не найдена — установите [.NET SDK](#) и перезапустите терминал.

Шаг 2. Проверка Git

git --version

Если Git не установлен — поставьте [Git for Windows](#).

При первом использовании Git на компьютере укажите своё имя и почту (один раз для всего компьютера):

git config --global user.name "Ваше Имя Фамилия"

git config --global user.email "your_email@example.com"

Шаг 3. Проверка VS Code и расширения

code --version

Откройте VS Code и убедитесь, что установлено расширение **C# Dev Kit** (Extensions → поиск "C# Dev Kit" → Install).

Часть 2. Создание репозитория

Шаг 4. Репозиторий на GitHub

Зайдите на github.com, нажмите **New repository**:

- Имя: algo-prog-course
- Видимость: **Private** (или Public — как требует преподаватель)
- ☒ Add a README file
- Add .gitignore → шаблон **VisualStudio**

Нажмите **Create repository**.

Шаг 5. Клонирование на компьютер

```
cd ~/Documents
```

```
git clone https://github.com/ВАШ_ЛОГИН/algo-prog-course.git
```

```
cd algo-prog-course
```

```
mkdir img
```

```
ls -la
```

 **СКРИНШОТ** — вывод `ls -la` в терминале. Сохраните в `img` с именем `step5Lab01_ФИО.png`.

Часть 3. Первая программа: Hello World

Шаг 6. Создание проекта Lab01

```
dotnet new console -n Lab01
```

```
cd Lab01
```

```
code .
```

Откройте `Program.cs` — там уже есть одна строка:

```
Console.WriteLine("Hello, World!");
```

Это называется **top-level statements** — начиная с C# 9, для простой программы не нужно писать `class Program { static void Main() { ... } }` вручную, компилятор достраивает всё сам. Мы будем пользоваться этим способом весь семестр — так короче.

Шаг 7. Запуск

Введите в терминале:

```
dotnet run
```

Должно вывести `Hello, World!`.

Шаг 8. Первый коммит

```
cd ..
```

```
git add .
```

```
git commit -m "Lab01: hello world"
```

```
git push
```

```
cd Lab01
```

Часть 4. Переменные и типы

Шаг 9. Переменные и интерполяция строк

Замените содержимое **Program.cs** на:

```
string myName = "Иван Иванов";
string groupName = "ИС-21";
int courseNumber = 2;
double averageGrade = 4.6;
bool isBudget = true;

Console.WriteLine("Знакомство");
Console.WriteLine($"Студент: {myName}");
Console.WriteLine($"Группа: {groupName}");
Console.WriteLine($"Курс: {courseNumber}");
Console.WriteLine($"Средний балл: {averageGrade}");
Console.WriteLine($"Бюджетное место: {isBudget}");
```

Замените `myName` и `groupName` на свои данные.

Что здесь нового:

- **string, int, double, bool** — типы переменных: текст, целое число, дробное число, логическое значение. Подробно разберём каждый в ПР2.
- **Интерполяция строк** `"..."` — способ подставить значение переменной прямо в текст. Фигурные скобки `{имя_переменной}` заменяются на значение.

Шаг 10. Запуск и проверка

dotnet run

Введите в терминале:

date

 **СКРИНШОТ** — вывод программы с датой. Сохраните в **img** с именем **step10Lab01_ФИО.png**.

Шаг 11. Коммит

cd ..

git add .

git commit -m "Lab01: variables and interpolation"

git push

cd Lab01

Часть 5. Арифметика на переменных

Шаг 12. Считаем площадь и периметр комнаты

Добавьте в конец **Program.cs**:

```

Console.WriteLine();
Console.WriteLine("Ремонт: комната");

double roomWidth = 3.5;
double roomLength = 4.2;

double roomArea = roomWidth * roomLength;
double roomPerimeter = (roomWidth + roomLength) * 2;

Console.WriteLine($"Ширина: {roomWidth} м, длина: {roomLength} м");
Console.WriteLine($"Площадь: {roomArea} кв.м");
Console.WriteLine($"Периметр: {roomPerimeter} м");

```

Запустите **dotnet run** и проверьте результат вручную на калькуляторе — должно сойтись.

Шаг 13. Считаем стоимость покупки

Добавьте **ниже**:

```

Console.WriteLine();
Console.WriteLine("Покупка ноутбука в рассрочку");

int laptopPrice = 65000;
int monthsCount = 12;
double interestRate = 0.08;

double totalWithInterest = laptopPrice * (1 + interestRate);
double monthlyPayment = totalWithInterest / monthsCount;

Console.WriteLine($"Цена ноутбука: {laptopPrice} руб.");
Console.WriteLine($"Итого с процентами: {totalWithInterest} руб.");
Console.WriteLine($"Платёж в месяц: {monthlyPayment} руб.");

```

Обратите внимание: `laptopPrice` — `int`, а `interestRate` — `double`. Когда в выражении смешиваются `int` и `double`, C# автоматически приводит результат к `double` — это называется **неявное приведение типов** (`implicit conversion`).

Шаг 14. Целочисленное деление — ловушка

Добавьте:

```

Console.WriteLine();
Console.WriteLine("Внимание: деление int");

int totalStudents = 25;
int groupsCount = 4;

```

```
int studentsPerGroupWrong = totalStudents / groupsCount;
double studentsPerGroupCorrect = (double)totalStudents / groupsCount;

Console.WriteLine($"25 / 4 как int: {studentsPerGroupWrong}");
Console.WriteLine($"25 / 4 как double: {studentsPerGroupCorrect}");
```

Запустите и посмотрите на разницу в результатах.

Почему так? Если **оба** операнда — `int`, результат деления тоже будет `int`, а дробная часть просто **отбрасывается** (не округляется!). Чтобы получить точный результат, нужно превратить хотя бы один операнд в `double` — это делается через `(double)` перед переменной, это называется **явное приведение типов** (`explicit cast`).

Введите в терминале:

date

 **СКРИНШОТ** — вывод всей программы целиком (комната + ноутбук + деление) с датой. Сохраните в `img` с именем `step14Lab01_ФИО.png`.

Шаг 15. Коммит

cd ..

git add .

git commit -m "Lab01: arithmetic with variables"

git push

cd Lab01

Часть 6. Строки, константы и комментарии

Шаг 16. Конкатенация против интерполяции

Добавьте в конец **Program.cs**:

```
Console.WriteLine();
Console.WriteLine("Способы собрать строку");

string firstName = "Анна";
string lastName = "Смирнова";

// Способ 1: конкатенация через оператор +
string fullNameConcat = firstName + " " + lastName;

// Способ 2: интерполяция через $"
string fullNameInterp = $"{firstName} {lastName}";
```

```
// Способ 3: метод string.Concat
string fullNameConcatMethod = string.Concat(firstName, " ", lastName);

Console.WriteLine(fullNameConcat);
Console.WriteLine(fullNameInterp);
Console.WriteLine(fullNameConcatMethod);
Console.WriteLine($"Все три строки равны: {fullNameConcat ==
fullNameInterp && fullNameInterp == fullNameConcatMethod}");
```

Что здесь нового:

- + между строками склеивает их (конкатенация). Работает, но с большим числом переменных код быстро становится нечитаемым.
- \$"..." — то же самое, но нагляднее. В этом курсе используем **интерполяцию** как основной способ.
- == между строками сравнивает их **содержимое** (в отличие от некоторых других языков, где == сравнивал бы адреса в памяти — но это тема будущего курса про ссылочные типы).

Шаг 17. Константы

Добавьте:

```
Console.WriteLine();
Console.WriteLine("Константы");

const double VatRate = 0.20;
const string CollegeName = "ВФ ВолГУ";

double productPrice = 1000;
double priceWithVat = productPrice * (1 + VatRate);

Console.WriteLine($"Учебное заведение: {CollegeName}");
Console.WriteLine($"Цена без НДС: {productPrice}, с НДС ({VatRate:P0}): {priceWithVat}");
```

Попробуйте добавить строку `VatRate = 0.18;` после объявления константы и запустить `dotnet run`.

Компилятор выдаст ошибку: **Cannot assign to variable 'VatRate' because it is read-only**. Удалите эту строку обратно.

const — переменная, значение которой задаётся один раз и **не может измениться** за всё время работы программы. Используйте `const` для значений, которые действительно постоянны (ставка налога, число π , название организации), — это

защищает от случайной перезаписи и делает код понятнее для того, кто его читает после вас.

Стиль именования: переменные — **camelCase** (productPrice), константы и типы — **PascalCase** (VatRate, Console). Мы будем придерживаться этого стиля весь курс.

Шаг 18. Комментарии

Комментарии — текст, который компилятор полностью игнорирует, но который помогает человеку понять код:

```
// Однострочный комментарий — до конца строки

/*
  Многострочный комментарий —
  может занимать несколько строк
*/

// TODO: часто используется, чтобы отметить недоделанный участок кода
```

Добавьте **хотя бы 3 содержательных комментария** к уже написанному коду в **Program.cs** — объясните словами, что делает каждый крупный блок (комната, ноутбук, строки, константы).

Комментарий вида `// прибавляем 1 к строке x = x + 1;` — плохой комментарий, он просто повторяет код. Хороший комментарий объясняет **зачем**, а не **что** — например, `// НДС фиксирован законом, поэтому const.`

Введите в терминале:

date

 **СКРИНШОТ** — вывод программы (строки + константы) с датой. Сохраните в `img` с именем **step18Lab01_ФИО.png**.

Шаг 19. Коммит

cd ..

git add .

git commit -m "Lab01: strings, constants, comments"

git push

cd Lab01

Часть 7. Итоговая практика: визитка студента

Шаг 20. Собираем всё вместе

Создайте **новый файл** внутри проекта Lab01 с именем Visitka.cs... на самом деле в top-level statements можно писать только в одном файле, поэтому вместо этого создадим отдельный проект.

```
cd ..
```

```
dotnet new console -n Visitka
```

```
cd Visitka
```

```
code .
```

Напишите в Program.cs программу «визитка студента», которая объединяет всё, что вы сегодня изучили. Программа должна содержать:

- **не менее 6 переменных** разных типов (string, int, double, bool);
- **хотя бы одну константу** (const) с логичным применением;
- **хотя бы один арифметический расчёт** с использованием переменных (например, сколько дней осталось до конца семестра, или средний балл по трём оценкам);
- **вывод через интерполяцию** \$"...", оформленный аккуратно, с разделителями (====, пустые строки);
- **не менее 3 комментариев**, поясняющих логику.

Пример возможного вывода (ваш дизайн может отличаться):

```
ВИЗИТНАЯ КАРТОЧКА СТУДЕНТА
ФИО:          Иванов Иван Иванович
Группа:        ИСП-251
Курс:          2
Специальность: 09.02.07

Средний балл за 3 работы: 4.33
Стипендия положена (>= 4.0): True

Учебных недель осталось в семестре: 16
```

Запустите:

dotnet run

 **СКРИНШОТ** — полный вывод программы «Визитка». Сохраните в `img` (в папке `Lab01`, для удобства) с именем **step20Lab01_ФИО.png**.

Шаг 21. Финальный коммит

cd ..

git add .

git commit -m "Lab01 final: student visitka program"

git push

Самостоятельные задания

Задание 1. Финансовый мини-расчёт ★

В проекте `Lab01` добавьте в конец `Program.cs` блок кода, который:

1. Хранит стипендию (`int scholarship`) и ежемесячные расходы на еду и проезд (`int monthlyExpenses`) в переменных.
2. Считает и выводит, сколько денег останется у студента к концу месяца (`scholarship - monthlyExpenses`).
3. Считает и выводит, сколько это составит за весь семестр (4 месяца) — используйте константу `const int MonthsInSemester = 4;`.

Закоммитьте:

git add .

git commit -m "Lab01 task 1: budget calculation"

git push

Задание 2. Конвертер температур ★★

Создайте ещё один новый проект `TempConverter`:

cd ~/Documents/algo-prog-course

dotnet new console -n TempConverter

Программа должна:

1. Хранить температуру в цельсиях в переменной `double celsius = 23.5;` (значение впишите своё).
2. Рассчитать и вывести эту же температуру в **Фаренгейтах** по формуле $F = C * 9/5 + 32$.

3. Рассчитать и вывести эту же температуру в **Кельвинах** по формуле $K = C + 273.15$.
4. Использовать **хотя бы одну константу** для чисел 9, 5, 32, 273.15 — по своему усмотрению, где это уместно.

Пример работы:

23.5°C = 74.3°F = 296.65K

Закоммитьте:

git add .

git commit -m "Lab01 task 2: temperature converter"

git push

Задание 3. Найди и исправь ошибку ★★★

В проекте **Lab01** создайте новый блок кода в конце **Program.cs**, намеренно воспроизведя ловушку целочисленного деления из Шага 14, но на новом примере:

1. Заведите **int totalMinutes = 500;** и **int minutesPerLesson = 45;**
2. Посчитайте, сколько **полных занятий** уместится (это должно быть целочисленное деление — здесь ошибки нет, так и задумано).
3. Посчитайте, сколько минут **останется** от последнего неполного занятия, используя оператор **%** (остаток от деления) — изучите его самостоятельно по документации или поиску в интернете.
4. Выведите результат в формате: 500 минут = 11 полных занятий + 5 минут.

Закоммитьте:

git add .

git commit -m "Lab01 task 3: modulo operator practice"

git push

Контрольные вопросы

Откройте **README.md** в корне репозитория **algo-prog-course** и добавьте раздел «**ПР1. Ответы на контрольные вопросы**». Ответьте на:

1. Чем компилируемый язык (C#) отличается от интерпретируемого (Python) с точки зрения того, когда обнаруживаются ошибки?
2. Что делает команда **dotnet run** на самом деле — из каких двух шагов она состоит?
3. Зачем нужны папки **bin/** и **obj/** и почему их не нужно хранить в Git?

4. В чём разница между `git commit` и `git push`?
5. Чем отличаются конкатенация строк через `+` и интерполяция через `$"..."`? Какой способ предпочтительнее и почему?
6. Что произойдёт, если разделить `int` на `int`, если результат должен быть дробным? Как это исправить?
7. Чем `const` отличается от обычной переменной? Приведите свой пример, когда стоит использовать `const`.
8. Что такое `top-level statements` и чем они отличаются от классического `class Program { static void Main() }`?